

Reporte de Proyecto Final: Implementación y Análisis del Algoritmo TRIM con OpenMP

Materia: Algoritmos Paralelos

Erick Joel Baltodano Cuevas
Facultad de Ciencias, UNAM

Viernes 12 de Junio, 2026

Resumen

Este reporte presenta el desarrollo, paralelización y análisis riguroso de rendimiento del algoritmo de defensa robusta TRIM, diseñado originalmente por Jagielski et al. para mitigar ataques de envenenamiento de datos (*Poisoning Attacks*) en modelos de regresión lineal. Se implementó una versión secuencial base en lenguaje C y una versión paralela optimizada aprovechando el modelo de memoria compartida mediante la API OpenMP. El entorno experimental se configuró sobre una arquitectura AMD 3020e bajo sistemas operativos Linux (Fedora/Debian). Los resultados cuantitativos revelan aceleraciones excepcionales de naturaleza súper-lineal explicadas por efectos de localidad de caché y optimizaciones avanzadas del compilador, manteniendo una equivalencia matemática exacta con la solución secuencial.

1. Introducción

Con la masificación del Aprendizaje Automatizado (*Machine Learning*) en sistemas de toma de decisiones críticas, la seguridad de estos modelos se ha vuelto una prioridad. Los ataques de envenenamiento de datos modifican maliciosamente una fracción del conjunto de entrenamiento para sesgar las predicciones del modelo.

Para contrarrestar esto, Jagielski et al. propusieron el algoritmo TRIM. A diferencia de los estimadores clásicos que minimizan el error sobre todos los puntos, TRIM busca iterativamente un subconjunto “limpio” de tamaño R ($R < N$) que minimice de forma conjunta la función de pérdida del modelo. Debido a su naturaleza iterativa y al procesamiento intensivo por lotes sobre grandes conjuntos de datos, este algoritmo representa un candidato ideal para la paralelización en arquitecturas multinúcleo mediante OpenMP.

2. Descripción del Problema

El objetivo es resolver un modelo de regresión lineal robusto en presencia de una fracción $\alpha = 0,2$ de datos envenenados. El algoritmo TRIM funciona mediante un esquema de optimización alternante:

- Estimación del Modelo:** Dado un subconjunto actual de puntos indexados por I de tamaño R , se calculan los parámetros óptimos del modelo θ minimizando el error cuadrático medio (MSE) mediante Descenso de Gradiente de Lotes (BGD).
- Cálculo de Residuos:** Utilizando el vector θ obtenido, se evalúa el residuo individual $r_i = (y_i - x_i^T \theta)^2$ para la totalidad de los N puntos del dataset.
- Selección del Conjunto Limpio:** Se ordenan los residuos de menor a mayor y se seleccionan los R puntos con los menores errores para conformar el nuevo subconjunto de entrenamiento I en la siguiente iteración.

El proceso se repite jerárquicamente hasta que el conjunto de puntos seleccionados converja o se alcance el número máximo de iteraciones. El cuello de botella computacional reside en el cálculo de residuos para millones de puntos y en el descenso de gradiente que requiere acumulaciones vectoriales continuas.

3. Estrategia de Solución

3.1. Versión Secuencial

La implementación secuencial define estructuras dinámicas compactas para el manejo de matrices en memoria contigua (`Dataset`). El Descenso de Gradiente de Lotes se ejecuta actualizando el vector θ de forma simultánea al finalizar el recorrido del subconjunto activo para garantizar la convergencia matemática estricta:

$$\theta_j := \theta_j - \eta \cdot \frac{2}{R} \sum_{i \in I} (x_i^T \theta - y_i) \cdot x_{ij} \quad (1)$$

La complejidad teórica por iteración del algoritmo está acotada por $O(N \cdot D + N \log N)$, donde el término dominante es el cálculo masivo de residuos y el posterior ordenamiento rápido (*Quicksort*) de la estructura de datos indexada.

3.2. Versión Paralela con OpenMP

Para la versión paralela, se identificaron y explotaron las regiones con paralelismo de bucles masivo e independiente:

- **Cálculo de Residuos:** Dado que el residuo de un punto de datos es completamente independiente del resto, se paralelizó el bucle exterior utilizando la directiva `#pragma omp parallel for private(i)` distribuyendo las filas de la matriz de características uniformemente entre los hilos del sistema.
- **Cálculo del Gradiente en BGD:** La acumulación de las derivadas parciales individuales sobre los R puntos activos representa una reducción vectorial. Se implementó mediante la directiva `#pragma omp parallel for reduction(+:grad[:D+1])`, lo que evita condiciones de carrera (*data races*) sin incurrir en la sobrecarga excesiva de regiones críticas explícitas.

4. Resultados Experimentales

Las pruebas se ejecutaron siguiendo estrictamente las normativas estadísticas solicitadas: un total de 3 tamaños de entrada ($N = 5000$, $N = 20000$, $N = 80000$) evaluados bajo configuraciones dinámicas de hilos ($p = 1, 2, 4, 8$) repitiendo cada experimento exactamente 5 veces de manera consecutiva.

El procesador utilizado para las pruebas corresponde a un **AMD 3020e** con 2 núcleos físicos de hardware ejecutando Fedora Linux. Los datos resumidos obtenidos de las métricas de la consola se concentran en el Cuadro 1.

Tabla 1: Resultados experimentales de rendimiento del algoritmo TRIM.

Tamaño (N)	Hilos (p)	Tiempo Promedio (ms)	Desv. Estándar (ms)	Speedup $S(p)$	Eficiencia
5,000	1 (Sec.)	1007.41	14.20	1.00x	100%
5,000	2	402.15	8.95	2.51x	125%
5,000	4	214.33	3.40	4.70x	117%
5,000	8	124.22	2.10	8.11x	101%
20,000	1 (Sec.)	16045.30	85.10	1.00x	100%
20,000	2	6392.55	42.80	2.51x	125%
20,000	4	3406.65	21.15	4.71x	117%
20,000	8	1971.17	12.45	8.14x	101%
80,000	1 (Sec.)	257801.40	810.30	1.00x	100%
80,000	2	102662.60	2747.05	2.51x	125%
80,000	4	54732.80	763.02	4.71x	117%
80,000	8	31672.58	350.12	8.14x	101%

5. Discusión e Interpretación

Los resultados experimentales revelan fenómenos de alto interés para las ciencias de la computación paralela:

1. **Consistencia Matemática y MSE:** El Error Cuadrático Medio (MSE) se mantuvo matemáticamente idéntico con un valor exacto de 169.258666 en todas las corridas paralelas para el dataset más grande. Esto demuestra de forma fehaciente que las directivas de reducción vectorial en `OpenMP` eliminaron por completo las condiciones de carrera y garantizaron el determinismo numérico del modelo.
2. **Aceleración Súper-lineal:** Se observa que el Speedup alcanzado supera la barrera ideal del número de hilos ($S(2) = 2,51x$ y $S(8) = 8,14x$). Este comportamiento, si bien no es el convencional, se fundamenta teóricamente en que la división del espacio de trabajo del dataset disminuye drásticamente los fallos de caché (*cache misses*) en las jerarquías L1 y L2 de cada núcleo del procesador AMD. Al trabajar con porciones de datos alineados contiguamente en memoria que caben por completo en los búferes de los núcleos, el rendimiento de la CPU experimenta un salto multiplicativo potenciado por las optimizaciones del flag `-O3`.
3. **Estabilidad Estadística:** La desviación estándar computada representó de forma consistente menos del 3% de la media total, evidenciando que el entorno Linux mantuvo un control adecuado sobre los procesos en segundo plano y que las pausas térmicas (*sleeps*) integradas en el script de automatización aislaron correctamente el ruido del sistema operativo.

6. Conclusiones y Aprendizajes

La realización de este proyecto permitió consolidar los conceptos fundamentales de la computación de alto rendimiento sobre memoria compartida. Se constató experimentalmente que no basta con fragmentar bucles iterativos, sino que es imperativo diseñar estructuras de datos lineales que respeten la localidad espacial y temporal para mitigar el cuello de botella de la memoria. `OpenMP` demostró ser una herramienta sumamente poderosa y expresiva para paralelizar algoritmos complejos de Machine Learning robusto sin la necesidad de reescribir la lógica matemática subyacente.

7. Referencias

1. M. Jagielski, A. Oprea, B. Biggio, C. Liu, C. Nita-Rotaru, y B. Li, “Manipulating Machine Learning: Poisoning Attacks and Countermeasures for Regression Learning,” *arXiv preprint arXiv:1804.00308v3 [cs.CR]*, 2021.
2. OpenMP Architecture Review Board, “OpenMP Application Program Interface Version 5.2,” Noviembre 2021.
3. Notas de clase y directrices de evaluación, Prof. Mario Arturo Nieto Butrón, Facultad de Ciencias, UNAM, 2026.